ICT374 Lab 3:
Shell Customization, File Access Permissions, Access Remote Hosts, Checking Process Status; C's Arrays and Pointers, Dynamic Memory Allocations.

Learning Objectives:

1. Be able to customize your shell using the shell's start-up files.
2. Understand the concept of file ownership and file and directory access permissions
3. Be able to protect your files from unauthorized access using `chmod` and `chown`.
4. Be able to use command `ip` to lookup the IP number assigned to your host..
5. Be able to use `ssh` to access remote hosts.
6. Be able to use `sftp` to transfer files between local computer and the remote host.
7. Be able to use `ps` command to find out various information about processes.
8. Be able to use `kill` command to terminate processes.
9. Learn to use arrays and pointers in C.
10. Understand and be able to pass arrays to functions in C.
11. Learn to use `malloc` to allocated memory at run-time.

Required Reading:

- Lecture Notes for Topic 3
- Rute: Ch 9, Ch 12, and Ch 14
- A mini manual for vi editor

Access to Software:

Face-to-Face students using the lab computers should copy your work from the lab computer to a USB drive, or to a online drive such as Microsoft OneDrive or Google Drive. This is necessary as you cannot retain anything on any one of the lab computers.

Online Flexible students should use Unix (Linux, macOS Darwin, or WSL on Windows 11) on their own computer.

Unix Exercises

The following exercises are designed to get familiar with Unix (Linux) systems. More information about these basic Unix commands is available from the relevant chapters in Rute. See the Required Readings.

1. **Shell Customization**

   The bash shell can be created as a login shell or non-login shell. When you login (by entering your username and password), a login shell is created. When you start a terminal, a non-login shell is started to run inside the terminal window.

   A bash login shell initializes itself by looking for files `~/.bash_profile`, `~/.bash_login`,

and `~/.profile`, in that order. The first of these files found is read and the commands in that file are executed.

Check your home directory to see which one of the above files exists under your home directory. You can customize your shell by adding commands into that file. Assume that of the three files listed above, only `.profile` exists (If none exists, you can create one). Type in the following commands into the file:

```
echo Today is $(date)
echo Welcome to Unix
```

and then test it by starting a new login shell with the following command:

```
bash -login
```

Exit the login shell by typing the command `exit`.

One of the useful customizations is change your shell prompt. This is done by adding the the following command to the above shell initialization file:

```
PS1="[\u@\h:\w] $ "
```

where `\u` stands for the username, `\h` stands for host name of the computer, and `\w` stands for the current directory. Of course you can change the shell prompt in whatever way you like, by redefining variable `PS1`.

Test the above shell prompt to see how it looks.

Another useful initialization is to add directories to your search path. For example, adding directory `/sbin` and the current directory `.` to the `PATH`:

```
PATH=$PATH:/sbin:.
```

Type the command

```
source  ~/.profile
```

to make the above change effective immediately. Now you should be able to run program `a.out` directly under the directory where the program is stored.

2. **File Ownership and File Access Permissions**

In Unix, each file is owned by one user and one user group. The command `ls -l` displays the ownership information. For example, the following output that this command might produce:

```
drwx------   2 xie      staff        512 Jul 17 17:32 ICT374
drwxr-xr-x   3 xie      staff        512 Feb 15  2006 public_html
-rw-r-----   1 xie      staff    2184881 Jul 11 08:36 www.tar.gz
```

shows that the three files/directories are all owned by user `xie` and group `staff`.

Each file or directory has three sets of permissions: one for the owner, one for the members of the group (minus the owner) and one for everyone else (ie any user who is not the owner nor a member of the group). Each set of permissions consists of read permission, write permission and execute permission. If you have execute permission on a

program (or command), you are allowed to run that program. If you have execute permission on a directory (in this case, the permission is called search permission), you are allowed to search the files under that directory.

The command `ls -l` uses rwx to denote read, write and execute permissions. It uses - to denote no permission. In each line, the first character tells you whether the file is a regular file or a directory ("-" means a regular file and "d" means a directory file). The next three characters denote the permissions for the owner. Characters 5-7 denote the permissions for members of the group except the owner. The characters 8-10 denote the permissions for everyone else.

For example, in the above output, files `ICT374` and `public_html` are directories, while file `www.tar.gz` is a regular file. The owner (ie the user "xie") has read and write permissions on file `www.tar.gz` (rw-). Any staff member except user "xie" can read the file (r--). Anyone else has no permission on the file (---).

You can change the access permissions of the files you own. You cannot change the permissions of files owned by others (however root can change the permissions of any file).

Use command `chmod` to change the access permissions. For example to make the file `www.tar.gz` readable and writable by everyone, you may use command

```
chmod 666 www.tar.gz
```

Check the manual page for `chmod` to see the details on how to use the command. Experiment with this command by setting different permissions on a file.

3. **Finding out the IP Number**

   You can use the following command to find out the IP number assigned to the network interface of your host:

   ```
   ip address
   ```

   or

   ```
   ip address show eth0
   ```

   If the shell complains "command not found", use its full path instead:

   ```
   /bin/ip address
   ```

4. **Accessing Remote Hosts**

   If you have an account on another computer (the remote host), and that computer runs a SSH-2 server, you can log in to the remote host using SSH client `ssh`. `ssh` is similar to `telnet`. The main difference is that `telnet` transmits data, including your password, in clear text, while `ssh` encrypts the data, therefore is more secure.

   You should have an account on `eris.ad.murdoch.edu.au`, which runs Debian Linux. Your username and password are your Murdoch Username and Murdoch Password. Now login to your eris account with following command:

   ```
   ssh your_username@eris.ad.murdoch.edu.au
   ```

Please note you should not do your systems programming exercises on eris. A buggy program can bring down the server computer with many people affected.

5. **File Transfer between Local Host and Remote Host**

   You can also transfer files between the local host and the remote host using `sftp` command. Its user interface is similar to that of `ftp`. The data and password are encrypted during the data transfer so it is more secure than `ftp`.

   Transfer a few files between your Linux virtual machine and eris with the following command:

   ```
   sftp your_username@eris.ad.murdoch.edu.au
   ```

   Note `sftp` has a fairly rich set of commands, such as

   - `cd` *`directory`* : set the current directory on the remote host
   - `lcd` *`directory`* : set the current directory on the local host
   - `pwd` : print the current directory on the remote host
   - `lpwd` : print the current directory on the local host
   - `ls` : list the current directory on the remote host
   - `lls` : list the current directory on the local host
   - `put` *`filename`* : transfer files from local host to remote host. Note you can use wildcards in the file name, such `Ex*.c` and `Ex*.?` .
   - `get` *`filename`* : transfer files from remote host to local host. Note you can use wildcards in the file name, such `Ex*.c` and `Ex*.?` .
   - `help` : get help

   Note: commands `ssh` and `sftp` are available on any Unix system. Now they are also available on Windows 10. Therefore you can also transfer files between your Windows 10 and the remote server `cedo`.

   Try it.

6. **Finding Out Process Status**

   One of the most important Unix commands is `ps`. With this command you can find out various information about processes running on your computer. Without options, this command displays the list of processes running under the current shell. Example:

   ```
   hong:~ $ps
     PID TTY          TIME CMD
   31587 pts/0    00:00:00 bash
   31601 pts/0    00:00:00 ps
   hong:~ $
   ```

   here `PID` is the process ID, a unique integer for each process. `TTY` tells you the terminal in

which the process runs. The command `tty` will display the name of your terminal. `TIME` tells you the CPU time used by the process. `CMD` shows the command that was used to invoke the process.

`ps` has many options. One often used is `-f`. With this option more information will be displayed:

```
hong:~ $ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
xie      31587 31586  0 21:43 pts/0    00:00:00 -bash
xie      31604 31587  0 21:52 pts/0    00:00:00 ps -f
hong:~ $
```

in the above example, `UID` shows the user that created the process. `PPID` is the parent PID. `STIME` is the starting time of the process.

Another useful option is `-l`:

```
hong:~ $ps -l
  F S   UID   PID  PPID  C PRI  NI ADDR    SZ WCHAN  TTY          TIME CMD
000 S  1002 31587 31586  0  72   0    -   502 wait4  pts/0    00:00:00 bash
000 R  1002 31611 31587  0  76   0    -   795 -      pts/0    00:00:00 ps
hong:~ $
```

in this example, `S` tells you the process state: `S` for sleep, `R` for running, `T` for stopped, `Z` for zombie or defunct processes. `UID` shows you the User ID number of the user who created the process. In Unix, each user has a username (such as `xie` in above example) and a integer uid (such as 1002 for user `xie`, and 0 for root). `PRI` tells you the scheduling priority of the process. It is between 0 (high) and 127 (low). `NI` is the *nice value*, which is used to change the priority indirectly. The nice value is between -20 (a greater chance of a higher priority) and 19 (a greater chance of lower priority).

To display all processes, use option `-e`. You can combine different options together:

```
hong:~ $ps -el
  F S   UID   PID  PPID  C PRI  NI ADDR    SZ WCHAN  TTY          TIME CMD
100 S     0     1     0  0  68   0    -   104 do_sel ?        00:00:12 init
040 S     0     2     1  0  69   0    -     0 contex ?        00:00:00 keventd
040 S     0     3     1  0  78  19    -     0 ksofti ?        00:00:00 ksoftirqd
040 S     0     4     1  0  69   0    -     0 kswapd ?        00:01:22 kswapd
040 S     0     5     1  0  69   0    -     0 bdflus ?        00:00:00 bdflush
040 S     0     6     1  0  69   0    -     0 kupdat ?        00:00:17 kupdated
040 S     0     7     1  0  69   0    -     0 jfsIOW ?        00:00:00 jfsIO
040 S     0     8     1  0  69   0    -     0 jfs_la ?        00:00:00 jfsCommit
040 S     0     9     1  0  69   0    -     0 jfs_sy ?        00:00:00 jfsSync
140 S     1    63     1  0  69   0    -   347 do_pol ?        00:00:00 rpc.portm
040 S     0    69     1  0  69   0    -   445 do_sel ?        00:03:25 syslogd
140 S     0    72     1  0  69   0    -   327 do_sys ?        00:00:00 klogd
140 S     0    74     1  0  68   0    -   443 do_sel ?        00:18:34 inetd
140 S     0    77     1  0  69   0    -   684 do_sel ?        00:01:05 sshd
040 S     0    86     1  0  69   0    -   332 do_pol ?        00:00:00 rpc.rquot
140 S     0    88     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0    90     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0    89     1  0  69   0    -     0 svc_re ?        00:00:00 lockd
040 S     0    91    89  0  69   0    -     0 rpciod ?        00:00:00 rpciod
040 S     0    92     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0    93     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0    94     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0    95     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
```

```
040 S     0    96     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0    97     1  0  69   0    -     0 svc_re ?        00:00:00 nfsd
040 S     0   100     1  0  69   0    -   341 do_pol ?        00:00:00 rpc.mount
140 S     0   103     1  0  69   0    -   347 do_sel ?        00:00:00 rpc.statd
140 S     0   105     1  0  69   0    -   452 do_sel ?        00:00:00 lpd
040 S     0   107     1  0  68   0    -   339 nanosl ?        00:00:00 crond
040 S     2   110     1  0  69   0    -   339 nanosl ?        00:00:00 atd
140 S     0   121     1  0  68   0    -   821 do_sel ?        00:00:00 httpd
100 S     0   127     1  0  79  19    -   859 nanosl ?        00:00:00 dnetc
000 S     0   131     1  0  69   0    -   449 wait4  ?        00:00:00 RunCache
140 S    99   143   121  0  69   0    -   859 wait_f ?        00:00:02 httpd
140 S    99   144   121  0  69   0    -   852 wait_f ?        00:00:02 httpd
140 S    99   145   121  0  69   0    -   859 wait_f ?        00:00:02 httpd
140 S    99   146   121  0  69   0    -   862 wait_f ?        00:00:02 httpd
140 S    99   147   121  0  69   0    -   855 wait_f ?        00:00:02 httpd
100 S     0   148     1  0  69   0    - 24123 wait_f ?        00:00:09 java
100 S     0   149     1  0  69   0    -   304 read_c tty1     00:00:00 agetty
000 S     0   150     1  0  69   0    -   304 read_c tty2     00:00:00 agetty
000 S     0   151     1  0  69   0    -   304 read_c tty3     00:00:00 agetty
000 S     0   152     1  0  69   0    -   304 read_c tty4     00:00:00 agetty
000 S     0   153     1  0  69   0    -   304 read_c tty5     00:00:00 agetty
000 S     0   154     1  0  69   0    -   304 read_c tty6     00:00:00 agetty
040 S     0   157   127  0  79  19    -   859 do_pol ?        00:00:00 dnetc
040 R     0   158   157 99  79  19    -   859 -      ?        123-19:52:07 dnetc
040 S     0   169   148  0  69   0    - 24123 do_pol ?        00:00:00 java
040 S     0   170   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   171   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   172   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
140 S     0   173   169  0  69   0    - 24123 nanosl ?        00:01:23 java
040 S     0   174   169  0  69   0    - 24123 nanosl ?        00:00:00 java
040 S     0   175   169  0  69   0    - 24123 nanosl ?        00:00:00 java
040 S     0   176   169  0  69   0    - 24123 nanosl ?        00:00:00 java
100 S    99   179   131  0  69   0    -  2465 do_pol ?        00:01:51 squid
040 S     0   180   169  0  69   0    - 24123 nanosl ?        00:00:00 java
000 S    99   181   179  0  69   0    -   300 pipe_w ?        00:00:00 unlinkd
040 S     0   183   169  0  69   0    - 24123 nanosl ?        00:00:00 java
040 S     0   184   169  0  69   0    - 24123 nanosl ?        00:00:00 java
140 S     0   185   169  0  69   0    - 24123 wait_f ?        00:00:00 java
040 S     0   191   169  0  69   0    - 24123 wait_f ?        00:00:00 java
040 S     0   192   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   193   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   194   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   195   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   196   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
040 S     0   197   169  0  69   0    - 24123 wait_f ?        00:00:00 java
140 S    99   708   121  0  69   0    -   862 wait_f ?        00:00:02 httpd
140 S    99   709   121  0  69   0    -   857 wait_f ?        00:00:02 httpd
140 S    99 32559   121  0  69   0    -   857 wait_f ?        00:00:02 httpd
140 S    99 12395   121  0  69   0    -   859 wait_f ?        00:00:02 httpd
140 S    99 17529   121  0  69   0    -   860 wait_f ?        00:00:02 httpd
140 S     0 23388   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
140 S     0 23389   169  0  69   0    - 24123 rt_sig ?        00:00:00 java
140 S     0 31584    77  0  69   0    -  1488 unix_s ?        00:00:00 sshd
140 S  1002 31586 31584  0  70   0    -  1336 do_sel ?        00:00:00 sshd
000 S  1002 31587 31586  0  72   0    -   502 wait4  pts/0    00:00:00 bash
000 R  1002 31612 31587  0  77   0    -   795 -      pts/0    00:00:00 ps
hong:~ $
```

As you can see most processes are run by the root and are not associated w with any terminal (`TTY=?`). Some of these are daemons, or servers.

Another option `-H` displays the process hierarchy, ie parent-child relationship. For

example:

```
S900432D@eris:~$ ps -efH | grep S900432D
S900432D 32628 32625  0 21:19 ?        00:00:00       sshd: S900432D@pts/0
S900432D 32629 32628  0 21:19 pts/0    00:00:00         -bash
S900432D   958 32629  0 22:20 pts/0    00:00:00           ps -efH
S900432D   959 32629  0 22:20 pts/0    00:00:00           grep S900432D
S900432D@eris:~$
```

The above command line consists of two commands `ps -efH` and `grep S900432D` connected by a pipe ("|"), which links the standard output of the first command with the standard input of the second command. The command `grep` picks up those input lines that contains the string "S900432D". The output in the above command line shows all processes started by the user "S900432D". It also reveals hierarchical structures of these processes.

Try all commands used in this questions and try to understand the output with the help of manual page of the `ps` command.

7. **Killing Processes**

   You can kill any process you own by sending the signal SIGKILL (signal 9). Firstly you need to find out the PID of the process you want to kill, then use the command

   ```
   kill -9 pid
   ```

   Note you cannot kill a process that is owned by other users. However root can kill any process.

Theory Exercises

1. What is the relationship between a program and its process? Why it is necessary to create a process in order to run a program?
2. What is the PCB of a process and what are stored in the PCB?
3. What is the dispatcher in an operating system and what does it do?
4. What are the states of a process? Draw a state transition diagram for a process. Explain what each of the following contains: ready queue and blocked queue.
5. What is the process image of a process? In a process image, which part is user visible and which part is only visible by the operating system kernel?
6. What types of events may cause the operating system to switch from one process to another process?
7. What would happen in a process switch? What is the role of CPU hardware and the operating system kernel respectively in a process switch?

C Programming Exercises

Note, all exercises must be done via the terminal commands, not GUI tools. Do not use IDE to build your programs. Do not use GUI file managers to manage your directories and files. Also,

each exercise should be in a separate directory, which itself, is under a directory called `lab04`

1. **Arrays and Pointers**

   In C, arrays and pointers can be used interchangeably in most situations. Let's look at the following example:

   ```
   int ai[10];
   int *pi;
   int i;

   for (i=0; i<10; ++i)
       ai[i] = i + 100;    // initialise each array element

   pi = ai;       // let pi point to the array
                  // alternatively, pi = &ai[0];

   for (i=0; i<10; ++i, ++pi)
       printf("a[%d]=%d\n", i, *pi);  // print each array element
                  // alternatively, printf("a[%d]=%d\n", i, ai[i]);
   ```

   in the above code, `int ai[10];` declares an array of 10 integers. The storage space for the array is allocated, but the array elements are not yet initialised.

   The code `int *pi;` declares a pointer variable that *can* hold an address of an integer. The pointer variable is allocated, however it does not contain any legitimate address yet.

   The `for` loop that follows initialises the 10 array elements with some values.

   Note, in C, the array name, such as `ai` represents the start address of the array. That address is same as `&ai[0]`. The statement `pi = ai;` assigns the start address of the array to pointer `pi`, so that it now points to the start of the array. In our next `for` loop, we print each array `ai[i]` via this pointer.

   Assuming `pi` contains the start address of array `ai`, Each of the following notations can be used to access *i*th array element:

   ```
   1)  n = ai[i];
   2)  n = pi[i];
   3)  n = *(ai + i);
   4)  n = *(pi + i);
   5)  pi = pi + i;
       n = *pi;
   6)  pi = ai + i;
       n = *pi;
   ```

   from the above notations, it is clear that arrays and pointers can almost be used interchangably. However we must understand that

   - With an array, its storage space is allocated automatically. If the array is declared outside of any function, the array size must be known at compile-time and its size remains fixed during the life of the process. If the array is declared inside a function, its size is fixed once the array is allocated until the function returns. As the storage space is allocated automatically, you do not need to explicitly allocate or deallocate space for the array (compared this to the array created using `malloc` in Exercise 3 below!).

- In ANSI C, also known as C90, the array size must be known at the compile-time. In C99 (a newer standard first published in 1999), you can declare a variable-length array as a function parameter or inside a function. However there are differences between variable-length array and dynamically allocated array using `malloc`. The former is allocated automatically in the stack but the later is allocated in the heap. The stack allocated variables disappear when the function returns but the heap allocated variables continue to exist until they are explicitly deallocated. The programmer must explicitly manage the dynamically allocated memory.

- With pointer, no storage space is allocated for the array. What is allocated is a pointer variable capable of storing a single memory address. In order to use it for an array, you must either make it point to an existing array, as shown in above example, or you dynamically allocate memory, using the function `malloc`, for the array and let the pointer point to the start of that memory space.

- While you can change the address stored in a pointer variable, so that it can point to different variables at different times, you cannot do the same with an array variable. Once declared, the array variable must be treated as an address constant.

Complete the above code and test it. Then modify the code using the six different notations (listed above) to access array elements and print these values.

2. **Pass Array to Functions**

   When passing an array variable to a function, such as this one:

   ```
   void fillArray(int array[], int arraySize);

   ....

   int ai[10];

   fillArray(ai, 10);

   ....
   ```

   we pass array `ai` to function `fillArray`. What exactly is passed to the function? The answer is a single memory address, not the 10 integers! This is why we can use the function to fill the array `ai` with 10 numbers.

   Complete the above function `fillArray` so that it reads `arraySize` number of integers from the user and fill the array with those numbers.

   Write a driver program to test the function with integer arrays of different sizes.

   Note the formal parameter `int array[]` in function `fillArray` can be changed to `int *array`. Verify this by modifying and testing your code.

3. **Dynamic Memory Allocations**

   In C Programming Exercise 1, we created an array of 10 integers. What if you do not know the size of the array you need to create at the compile-time? This can occur when the array size can only be determined from the user input at the run-time? For problems of this nature, you will have to use dynamic arrays. The following example demonstrates

how to create an array of `n` integers, where `n` is a variable:

```
int *ai;
int i;
int n;

n = 10;    // note this value can change at run-time
ai = malloc(sizeof(int) * n);

for (i=0; i<10; ++i)
    ai[i] = i + 100;    // initialise each array element

for (i=0; i<10; ++i)
    printf("a[%d]=%d\n", i, ai[i]);  // print each array element

free(ai);  // once the array is no longer needed, you must free it
```

Note the dynamically allocated memory must be deallocated (freed) once it is no loner needed. Otherwise you will have memory leaks - a very nasty bug that is hard to debug!

Convert the above code into a complete program with one change: the size of the array must be read from the user.

4. **Creating a Dynamic Array from a Function**

   The dynamic array depicted in the previous exercise is allocated in the heap, not in the stack as are local variables. Its life span is not affected by entering or leaving a function.

   Modify the C Programming Exercise 2 so that the function not only fills the array - it will create the dynamic array first. It should also return to its caller a pointer to the dynamically allocated array. The function should look like:

```
int *getArray(int arraySize)
{
     int *array;

     array = allocate memory for "arraySize" integers

     fill the array with some numbers

     return array;
}
```

   The above function can be called this way:

```
int *pi;

prompt user to type the array size n

pi = getArray(n);

.....
```

   Do not forget to deallocate the array!

The Weekly Lab

Submit the following exercises:

1. Theory Exercise 7.
2. C Program Exercise 1.
3. C Program Exercise 2.
4. C Program Exercise 4.

This weekly lab must be submitted to the LMS. The deadline of submission is given in the Unit Information Page of the LMS. You must adhere to the instructions in the document titled The Requirements for Documenting and Submitting Weekly Labs when preparing and submitting a weekly lab.

Dr Hong Xie
Unit Coordinator

Last update: Thursday, 14 August 2025

Document author: Dr. H. Xie, Unit Coordinator, ICT374 Operating Systems and Systems Programming
Last Modified: Friday, 15-Aug-2025 10:26:50 AWST
Disclaimer & Copyright Notice © 2026 Murdoch University
This document is relevant for 2025 only